



Giai doan 1 — Huong dan trien khai doc lap (4 Module)



Phien ban 1.0 | Thang 5/2025 | Fintech Learning Project

Du an	Auth & Identity Service (doc lap)
Muc tieu	Hoc JWT + Refresh Token + OAuth2 + 2FA truoc khi tích hop vao Wallet
Tech Stack	Spring Boot 3.3 MySQL 8 Redis 7 Java 21
Thu tu hoc	Module 1 -> 2 -> 3 -> 4 (co the song song M3 va M4)
Phien ban	1.0 — Draft

1. TONG QUAN VA MUC TIEU

1.1 Tai sao hoc doc lap truoc?

Khi tích hợp Auth vào một project có sẵn, bạn sẽ gặp nhiều lỗi khó debug vì chưa hiểu cơ chế bên trong. Học đọc lập cho phép bạn: (1) kiểm soát toàn bộ luồng, (2) debug từng bước mà không bị nhiễu bởi code nghiệp vụ, (3) hiểu chính xác tại sao mọi thứ hoạt động trước khi kết hợp chung lại.

1.2 Thu tu 4 module va moi lien he

M a	Tên module	Nội dung chính	Thu viện chính	Phụ thuộc
M 1	JWT Co Ban	Register, login, JwtFilter, bảo vệ endpoint	JWT, Spring Security 6	Bắt buộc trước
M 2	Refresh Token	Access token ngắn hạn, refresh token dài hạn, Redis blacklist	Redis, Spring Data Redis	Cần M1 xong
M 3	OAuth2 Google	Social login, bind account, issue JWT	Spring OAuth2 Client	Đọc lập, cần M1
M 4	TOTP 2FA	Google Authenticator, QR code, 2-step login	google-auth library	Cần M1+M2

1.3 Chuan bi moi truong (mot lan duy nhat)

Phan mem can cai dat

Công cụ	Nguồn	Ghi chú
JDK 21	oracle.com/java hoặc sdkman	<code>java -version</code> kiểm tra
Maven 3.9+	maven.apache.org	<code>mvn -version</code>
MySQL 8	dev.mysql.com hoặc Docker	Chạy: <code>docker run -d -p 3306:3306 -e MYSQL_ROOT_PASSWORD=root mysql:8</code>
Redis 7	redis.io hoặc Docker	Chạy: <code>docker run -d -p 6379:6379 redis:7</code>
IntelliJ IDEA	jetbrains.com (Community OK)	Plugin: Spring Boot, Lombok
Postman	postman.com	Để test API thủ công
TablePlus / DBeaver	Tùy chọn	Xem dữ liệu MySQL và Redis

Cau truc project auth-service (Spring Initializr)

- Group: com.fintech | Artifact: auth-service | Java 21 | Maven
- Dependencies: Spring Web, Spring Data JPA, Spring Security, MySQL Driver, Lombok, Validation
- Them thu cong vao pom.xml: jjwt-api, jjwt-impl, jjwt-jackson (M1), spring-boot-starter-data-redis (M2)
- Package structure: controller / service / impl / repository / domain / dto / config / filter / exception

Database can tao

```
CREATE DATABASE auth_db CHARACTER SET utf8mb4;
```

2.1 Muc tieu module

- Hieu cau truc JWT: header.payload.signature va cach ky bang HMAC-SHA256
- Xay dung luong Register va Login tra ve JWT
- Viet JwtFilter: moi request den /api/** phai co Bearer token hop le
- Hieu tai sao Spring Security can UserDetailsService va no hoat dong the nao

2.2 Cac phu thuoc can them (pom.xml)

```
<dependency> <groupId>io.jsonwebtoken</groupId> <artifactId>jjwt-api</artifactId>  
<version>0.12.6</version> </dependency> <dependency> <groupId>io.jsonwebtoken</groupId>  
<artifactId>jjwt-impl</artifactId> <version>0.12.6</version> <scope>runtime</scope>  
</dependency> <dependency> <groupId>io.jsonwebtoken</groupId>  
<artifactId>jjwt-jackson</artifactId> <version>0.12.6</version> <scope>runtime</scope>  
</dependency>
```

2.3 Database schema

Column	Kieu / Constraint	Ghi chu
id	BIGINT AUTO_INCREMENT	PRIMARY KEY
email	VARCHAR(100) UNIQUE NOT NULL	Dung lam username de login
password_hash	VARCHAR(255) NOT NULL	BCrypt hash, KHONG luu plain text
full_name	VARCHAR(100)	
role	ENUM('USER','ADMIN') DEFAULT 'USER'	
enabled	BOOLEAN DEFAULT TRUE	False khi bi lock
created_at	DATETIME NOT NULL	

2.4 Cac buoc trien khai chi tiet

B1	User entity + Repository	Tao User entity map vao bang users. UserRepository extends JpaRepository. Method: Optional findByEmail(String email).
B2	PasswordEncoder Bean	Tao @Bean BCryptPasswordEncoder trong SecurityConfig. KHONG tao UserDetailsService truoc buoc nay de tranh circular dependency.

B3	UserDetailsService	Implement UserDetailsService.loadUserByUsername(email): tìm User trong DB, tra về UserDetails. Nếu không tìm thấy -> throw UsernameNotFoundException.
B4	JwtUtil	@Component chứa: SECRET_KEY (min 256-bit), EXPIRATION_MS (900000 = 15 phút). Methods: generateToken(UserDetails), extractUsername(token), isValidToken(token, UserDetails).
B5	JwtFilter	extends OncePerRequestFilter. Logic: (1) Lấy header Authorization, (2) Trích Bearer token, (3) extractUsername, (4) load UserDetails, (5) isValidToken -> set SecurityContext, (6) filterChain.doFilter().
B6	SecurityFilterChain	Config: disable CSRF (REST API), sessionManagement = STATELESS, permit /auth/**, protect /api/**. Thêm jwtFilter trước UsernamePasswordAuthenticationFilter.
B7	AuthController — Register	POST /auth/register: nhận RegisterRequest(email, password, fullName). Kiểm tra email tồn tại chưa. Hash password. Lưu User. Trả 201 + thông tin cơ bản (KHÔNG trả password).
B8	AuthController — Login	POST /auth/login: dùng AuthenticationManager.authenticate(). Nếu sai -> throw 401. Nếu đúng -> generateToken(userDetails) -> trả LoginResponse{token, expiresIn}.
B9	Test endpoint bảo vệ	Tạo GET /api/hello tra về "Hello {email}". Gọi không có token -> 403. Gọi có token -> 200 + tên user.

2.5 application.yml cho Module 1

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/auth_db?createDatabaseIfNotExist=true
    username: root
    password: root
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
  app:
    jwt:
      secret: "môt-chuôi-bi-mat-it-một-256-bit-de-ky-HS256-không-dùng-production"
      expiration-ms: 900000 # 15 phút
```

2.6 Test cases Module 1

ID	Mô tả	Input / Điều kiện	Kết quả mong đợi	Pass/Fail
TC-1-01	Register thành công	POST /auth/register {email, password, fullName}	HTTP 201, trả UserResponse không có password	PASS
TC-1-02	Register email trùng	Gọi 2 lần cùng email	HTTP 409 "Email đã tồn tại"	PASS
TC-1-03	Register email sai định dạng	email = "không-phải-email"	HTTP 400 validation error	PASS
TC-1-04	Login đúng	POST /auth/login {email, password}	HTTP 200, LoginResponse có token string	PASS

TC-1-05	Login sai password	password = "sai"	HTTP 401 "Thông tin đăng nhập không chính xác"	PASS
TC-1-06	Goi /api/hello không có token	GET /api/hello (không có Authorization header)	HTTP 403 Forbidden	PASS
TC-1-07	Goi /api/hello có token hợp lệ	Authorization: Bearer	HTTP 200 "Hello user@example.com"	PASS
TC-1-08	Goi /api/hello với token hết hạn	Token đã hết hạn 15 phút	HTTP 403 và message lỗi trong body	PASS
TC-1-09	Goi /api/hello với token giả mạo	Token bị chỉnh sửa 1 ký tự	HTTP 403 (signature invalid)	PASS

DIEM HIEU QUAN TRONG M1: Sau khi viết xong, mở file JWT đã gen ra, decode trên jwt.io. Kiểm tra: header.alg = "HS256", payload có sub (email), iat, exp. Hieu rõ 3 phần này mới sang M2.

3.1 Mục tiêu module

- Hiểu tại sao access token ngắn hạn (15p) mà không cần login lại mỗi 15 phút
- Xây dựng cơ chế refresh token 7 ngày lưu trong Redis
- Implement Token Rotation: mỗi lần refresh -> cấp token mới, hủy token cũ
- Logout: revoke refresh token -> hoàn toàn đăng xuất
- Blacklist access token còn hạn nhưng cần hủy khẩn cấp

3.2 Dependency bổ sung

```
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-redis</artifactId> </dependency>
```

3.3 Cấu trúc dữ liệu trong Redis

Redis Key Pattern	Kiểu	TTL	Ý nghĩa
refresh:{userId}	String	7 ngày	Lưu refresh token hiện tại của user. Key theo userId để dễ revoke.
blacklist:{jti}	String	Còn lại của access token	Blacklist access token bị thu hồi. jti = JWT ID claim.
locked:{email}	String	15 phút	Mở rộng: đếm số lần login sai (sử dụng ở M4/tích hợp).

TAI SAO KEY THEO userId THAY VÌ TOKEN? Vì nếu lưu theo token, khi user đổi password hoặc logout toàn bộ thiết bị, bạn chỉ cần xóa 1 key redis:{userId} là revoke ngay, không cần biết token là gì.

3.4 Các bước triển khai chi tiết

B1	Thêm jti vào JWT	Trong generateToken(): thêm .id(UUID.randomUUID().toString()) -> mỗi token có jti duy nhất phục vụ blacklist.
B2	RefreshTokenService	generateRefreshToken(userId): tạo UUID, lưu vào Redis key "refresh:{userId}", TTL 7 ngày, trả về token string.
B3	Cập nhật Login response	Sau login thành công: gọi generateRefreshToken(user.getId()). Trả LoginResponse{accessToken, refreshToken, expiresIn:900}.

B4	Endpoint POST /auth/refresh	Nhan {refreshToken}. (1) Tim userId tu Redis (scan hoac luu mapping). (2) Xac minh token khop. (3) Xoa token cu. (4) Tao access token moi + refresh token moi. (5) Tra response.
B5	Token Rotation	Trong buoc refresh: deleteByKey("refresh:{userId}"), sau do generateRefreshToken(userId) -> luu token moi. Refresh token cu khong dung duoc nua.
B6	Endpoint POST /auth/logout	Nhan Authorization header. (1) Trich jti tu access token. (2) Tinh TTL con lai cua token. (3) Set Redis "blacklist:{jti}" = "revoked" voi TTL do. (4) Xoa "refresh:{userId}". (5) Tra 200.
B7	Cap nhat JwtFilter	Sau buoc isValid(): kiem tra them Redis.hasKey("blacklist:{jti}"). Neu co -> token bi revoke -> tra 403 ngay ca khi chua het han.
B8	application.yml bo sung	spring.data.redis.host=localhost, spring.data.redis.port=6379. app.jwt.refresh-expiration-ms=604800000 (7 ngay).

3.5 Test cases Module 2

ID	Mo ta	Input / Dieu kien	Ket qua mong doi	Pass/ Fail
TC-2-01	Login tra ve ca access + refresh token	POST /auth/login	HTTP 200, co 2 truong accessToken va refreshToken	PASS
TC-2-02	Refresh token hop le	POST /auth/refresh {refreshToken}	HTTP 200, accessToken moi + refreshToken moi (khac cai cu)	PASS
TC-2-03	Refresh token cu sau khi da refresh	Dung refreshToken cu goi /auth/refresh lan 2	HTTP 401 "Refresh token khong hop le"	PASS
TC-2-04	Logout thanh cong	POST /auth/logout voi Bearer token	HTTP 200. Goi /api/hello bang token cu -> 403	PASS
TC-2-05	Access token bi blacklist	Logout xong, goi /api/hello bang token vua logout	HTTP 403 du token chua het han	PASS
TC-2-06	Refresh token sau khi logout	Logout xong, goi /auth/refresh bang refreshToken cu	HTTP 401 (refresh token da bi xoa khoi Redis)	PASS
TC-2-07	Redis down: login van hoat dong?	Stop Redis, thu login	HTTP 500 hoac degraded (tuy config). Can ghi nhan behavior.	PASS

4.1 Muc tieu module

- Cho phép đăng nhập bằng tài khoản Google mà không cần đăng ký
- Hiểu luồng OAuth2 Authorization Code: redirect -> Google -> callback -> JWT
- Xử lý bind account: nếu email Google đã có trong DB -> link vào account cũ
- Issue JWT sau OAuth2 success, trả về cho frontend giống login thường

4.2 Chuẩn bị trên Google Cloud Console

B1	Tạo project	Vào console.cloud.google.com -> New Project -> đặt tên "auth-service-dev"
B2	Enable API	APIs & Services -> Enable APIs -> tìm "Google+ API" hoặc "Google Identity" -> Enable
B3	OAuth consent	OAuth consent screen -> External -> điền App name, email -> Save
B4	Tạo credentials	Credentials -> Create Credentials -> OAuth 2.0 Client IDs -> Web application
B5	Redirect URI	Thêm vào Authorized redirect URIs: http://localhost:8080/login/oauth2/code/google
B6	Lấy key	Copy Client ID và Client Secret -> thêm vào application.yml (KHÔNG commit lên Git)

4.3 Dependency và application.yml

```
# pom.xml <dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-oauth2-client</artifactId> </dependency> #
application.yml spring: security: oauth2: client: registration: google: client-id:
${GOOGLE_CLIENT_ID} client-secret: ${GOOGLE_CLIENT_SECRET} scope: email, profile
```

4.4 Các bước triển khai chi tiết

B1	Bổ sung User entity	Thêm trường: googleId VARCHAR(100) UNIQUE NULL, authProvider ENUM("LOCAL","GOOGLE") DEFAULT "LOCAL".
B2	CustomOAuth2UserService	implements OAuth2UserService. Lấy email từ attributes. Tìm User theo email trong DB. Nếu chưa có -> tạo User mới với authProvider=GOOGLE. Nếu có nhưng authProvider=LOCAL -> throw "Email đã đăng ký bằng mật khẩu".
B3	OAuth2AuthenticationSuccessHandler	implements AuthenticationSuccessHandler. Lấy CustomUserDetails từ authentication. Gọi generateToken() -> tạo JWT. Redirect về frontend URL kèm token: http://localhost:3000/oauth/callback?token=xxx

B4	OAuth2AuthenticationFailureHandler	implements AuthenticationFailureHandler. Redirect ve <code>http://localhost:3000/login?error=oauth_failed</code> .
B5	Cap nhat SecurityFilterChain	<code>.oauth2Login(oauth2 -> oauth2 .userInfoEndpoint(...).successHandler(...).failureHandler(...))</code>
B6	Test thu cong	Mo trinh duyet: <code>http://localhost:8080/oauth2/authorization/google</code> -> Se redirect sang trang dang nhap Google -> sau khi dang nhap -> redirect ve <code>localhost:3000</code> kem token.
B7	Xu ly token o "frontend"	Trong scope nay, dung Postman hoac tao HTML don gian de bat token tu URL param. Parse JWT tren <code>jwt.io</code> de xac nhan payload chinh xac.

4.5 Test cases Module 3

ID	Mo ta	Input / Dieu kien	Ket qua mong doi	Pass/ Fail
TC-3-01	Login Google lan dau (chua co account)	Trinh duyet -> <code>/oauth2/authorization/google</code> -> chon tai khoan	Redirect ve frontend kem token. DB co user moi voi <code>authProvider=GOOGLE</code>	PASS
TC-3-02	Login Google lan 2 (da co account)	Cung tai khoan Google, login lan 2	Redirect kem token. DB khong tao them user moi	PASS
TC-3-03	JWT tu OAuth2 hoat dong voi <code>/api/**</code>	Dung token tu OAuth2 goi GET <code>/api/hello</code>	HTTP 200, tra ten user Google	PASS
TC-3-04	Email Google trung voi LOCAL account	Dang ky LOCAL truoc, sau do login Google cung email do	HTTP 400 "Email nay da dang ky bang mat khau"	PASS
TC-3-05	Truy cap truc tiep <code>/login/oauth2/code/google</code>	GET <code>/login/oauth2/code/google</code> (khong qua Google)	HTTP 400 hoac redirect loi (khong co state param)	PASS

5.1 Muc tieu module

- Hieu TOTP: Time-based One-Time Password, cach tinh 6 chu so moi 30 giay
- Cho phep user bat/tat 2FA tu nguyen (khong bat buoc toan bo)
- Implement 2-step login: buoc 1 verify password, buoc 2 verify TOTP code
- Sinh backup codes de dung khi mat thiet bi

5.2 TOTP hoat dong the nao

TOTP (RFC 6238) dua tren cong thuc: $OTP = HOTP(secret, T)$ trong do $T = \text{floor}(\text{unix_time} / 30)$. Ca server va app dieu co cung secret -> tinh ra cung OTP tai cung thoi diem. Secret duoc ma hoa Base32, truyen den app qua QR code (otpauth:// URI). Server chap nhan +-1 window (30 giay) de bu clock skew.

5.3 Dependency can them

```
<!-- TOTP library --> <dependency> <groupId>com.warrenstrange</groupId>
<artifactId>googleauth</artifactId> <version>1.5.0</version> </dependency> <!-- QR code
generation --> <dependency> <groupId>com.google.zxing</groupId>
<artifactId>core</artifactId> <version>3.5.3</version> </dependency> <dependency>
<groupId>com.google.zxing</groupId> <artifactId>javase</artifactId>
<version>3.5.3</version> </dependency>
```

5.4 Bo sung Database

Column	Kieu	Y nghia
totp_secret	VARCHAR(100) NULL	Base32 secret dung cho TOTP, NULL neu chua bat 2FA
totp_enabled	BOOLEAN DEFAULT FALSE	Flag 2FA da kich hoat hay chua
backup_codes	TEXT NULL	JSON array 8 ma du phong, ma hoa bang BCrypt

5.5 Cac buoc trien khai chi tiet

B1	POST /auth/2fa/setup	User goi (can Bearer token). Server: (1) Tao GoogleAuthenticator, goi ga.createCredentials() ra secret. (2) Luu secret vao user.totpSecret (chua enabled). (3) Tao QR URL: otpauth://totp/AuthService:{email}?secret={base32}&issuer;=AuthService. (4) Dung ZXing viet QR URL thanh PNG base64. (5) Tra ve {qrCodeBase64, secret}.
-----------	----------------------	--

B2	POST /auth/2fa/verify-setup	User nhập mã 6 chữ số từ app sau khi quét QR. Server: (1) Lấy secret từ DB. (2) GoogleAuthenticator.authorize(secret, code). (3) Nếu đúng -> set <code>totp_enabled=true</code> . (4) Sinh 8 backup codes ngẫu nhiên, hash bằng BCrypt, lưu JSON vào <code>backup_codes</code> . (5) Trả về danh sách backup codes để user lưu (chỉ hiển thị 1 lần).
B3	Cap nhật Login flow (2 bước)	Bước 1: POST /auth/login -> verify password -> nếu <code>totp_enabled=false</code> : trả full JWT như cũ. Nếu <code>totp_enabled=true</code> : trả <code>PartialToken{partialToken, requiresTwoFactor:true}</code> . <code>PartialToken</code> là JWT ngắn hạn (5 phút) chỉ chứa claim <code>"stage":"partial"</code> .
B4	POST /auth/2fa/validate	Nhận {partialToken, code}. (1) Verify partialToken và lấy email. (2) Tìm user, lấy secret. (3) Kiểm tra code bằng GoogleAuthenticator.authorize() hoặc là backup code. (4) Nếu đúng -> issue full JWT + refresh token. Nếu sai -> 401.
B5	JwtFilter phân biệt partial token	Trong JwtFilter: nếu token có claim <code>stage="partial"</code> -> chỉ cho phép truy cập /auth/2fa/**. Gọi /api/** bằng partial token -> 403 "Chưa hoàn thành xác thực 2FA".
B6	Backup code logic	Khi user nhập backup code: quét danh sách, dùng BCrypt.matches(input, hash). Nếu khớp -> xóa code đó khỏi danh sách, cập nhật DB. Mỗi backup code chỉ dùng 1 lần.
B7	POST /auth/2fa/disable	Yêu cầu Bearer token và xác nhận password lần nữa. Xóa <code>totpSecret</code> , đặt <code>totp_enabled=false</code> , xóa <code>backup_codes</code> .

5.6 Test cases Module 4

ID	Mô tả	Input / Điều kiện	Kết quả mong đợi	Pass/Fail
TC-4-01	Setup 2FA thành công	POST /auth/2fa/setup với Bearer token	HTTP 200, trả <code>qrCodeBase64</code> và secret. DB lưu secret nhưng <code>totp_enabled=false</code>	PASS
TC-4-02	Verify setup 2FA đúng mã	POST /auth/2fa/verify-setup {code: mã từ app}	HTTP 200, <code>totp_enabled=true</code> trong DB, trả 8 backup codes	PASS
TC-4-03	Verify setup 2FA sai mã	POST /auth/2fa/verify-setup {code: "000000"}	HTTP 401 "Mã xác thực không chính xác"	PASS
TC-4-04	Login khi 2FA chưa bật	POST /auth/login (user chưa enable 2FA)	HTTP 200, trả full JWT như cũ (không có partial token)	PASS
TC-4-05	Login khi 2FA đã bật (bước 1)	POST /auth/login (user đã enable 2FA)	HTTP 200, trả {partialToken, requiresTwoFactor:true}	PASS

TC-4-06	Goi /api/hello bang partial token	Authorization: Bearer	HTTP 403 "Chua hoan thanh xac thuc 2FA"	PASS
TC-4-07	Validate 2FA dung ma (buoc 2)	POST /auth/2fa/validate {partialToken, code}	HTTP 200, tra full JWT + refreshToken	PASS
TC-4-08	Dung backup code thay cho TOTP	POST /auth/2fa/validate {code: backup_code}	HTTP 200, backup code bi xoa khoi DS (1 lan dung)	PASS
TC-4-09	Dung lai backup code da dung	Dung cung backup code lan 2	HTTP 401 (code khong con hop le)	PASS
TC-4-10	Tat 2FA	POST /auth/2fa/disable voi mat khau xac nhan	HTTP 200, totp_enabled=false, secret bi xoa	PASS

6. KIEM TRA TOAN DIEN VA CHECKLIST

6.1 Luong end-to-end day du

Sau khi hoan thanh 4 module, chay luong sau de xac nhan he thong hoat dong dung:

L1	Register LOCAL	POST /auth/register -> 201, user trong DB voi password hash
L2	Login -> lay token	POST /auth/login -> accessToken + refreshToken
L3	Goi API	GET /api/hello voi Bearer accessToken -> 200 + ten user
L4	Refresh token	POST /auth/refresh -> accessToken moi, refreshToken moi
L5	Setup 2FA	POST /auth/2fa/setup -> quet QR, verify, nhan backup codes
L6	Login 2 buoc	Login -> partial token -> validate TOTP -> full JWT
L7	OAuth2 login	Trinh duyet -> Google -> callback -> JWT token
L8	Logout	POST /auth/logout -> goi lai API bang token cu -> 403

6.2 Checklist bao mat bat buoc

- ✓ Password KHONG BAO GIO duoc luu plain text (phai BCrypt hash)
- ✓ JWT secret it nhat 256-bit, KHONG hardcode trong code (dung env var)
- ✓ Access token TTL <= 15 phut. KHONG dung token song mai mai
- ✓ Moi endpoint can auth phai tra 401/403 neu khong co / sai token
- ✓ KHONG tra password_hash trong bat ky response nao
- ✓ Partial token chi dung duoc cho /auth/2fa/**, khong the goi API khac
- ✓ Backup codes hash bang BCrypt, KHONG luu plain text
- ✓ CSRF disable hop le cho REST API (stateless session)
- ✓ HTTPS bat buoc cho production (trong dev OK voi localhost)
- ✓ Khong log JWT token vao console (security leak)
- ✓ Redis connection co password trong production

6.3 Loi thuong gap va cach fix

Loi thuong gap	Cach xu ly
Circular dependency: SecurityConfig -> UserDetailsService -> PasswordEncoder -> SecurityConfig	Tach PasswordEncoder Bean ra mot config riêng hoac dung @Lazy

JWT verify loi "signature does not match": secret khác giữa generate và verify	Dam bao dung cung 1 @Value inject, không tạo Key object nhiều nơi
OAuth2 loi "redirect_uri_mismatch"	URI trong Google Console phải giống hết URI trong application.yml, kể cả trailing slash
TOTP loi "Invalid code" do nhập dung: đồng hồ máy lệch	TOTP chấp nhận +/- 1 window. Kiểm tra System.currentTimeMillis() và NTP sync
Partial token access /api/** được (quen filter)	Trong JwtFilter kiểm tra claim "stage". Nếu stage=partial và path không bắt đầu /auth/2fa -> reject
Redis connection refused: quên chạy Redis	docker run -d -p 6379:6379 redis:7 hoặc redis-server &

6.4 Sau giai đoạn 1: chuẩn bị cho tích hợp

Sau khi hoàn thành 4 module và pass toàn bộ test case ở trên, bạn sẵn sàng sang Giai đoạn 2 (tích hợp vào Digital Wallet API). Nhưng thu cần mang theo:

- JwtUtil.java: class tiện ích generate/verify JWT
- JwtFilter.java: OncePerRequestFilter đã chạy on
- SecurityConfig snippet: SecurityFilterChain có cấu hình dung
- application.yml phần jwt: secret và expiration
- Hiểu rõ: cách lấy userId từ SecurityContext trong bất kỳ method nào
- Hiểu rõ: tại sao cần kiểm tra account ownership (user A không được dùng account của user B)

MỤC TIÊU CUỐI: Sau giai đoạn 1, bạn phải có thể giải thích thành thạo với interviewer: "Tại sao access token ngắn hạn?", "Tại sao cần refresh token?", "Nếu token bị đánh cắp, cần làm gì?" — Đây là các câu hỏi phổ biến ở Fintech interview.